

Topic:- BFS

Problem:-

1.01 Matrix

2.Minimum Cost to Make at Least One Valid Path in a Grid

Problem-1:

It can be solved using three approaches.

1.Multi Source BFS

2.Brute Force

3.Dynamic Programming

APPROACH 1

Multi Source BFS -

Definition:Multi-Source BFS is a variation of Breadth-First Search (BFS) where instead of starting from one source (node), we start from multiple sources at the same time.

Code:

```
class Solution {
public:
    vector<vector<int>> updateMatrix(vector<vector<int>>& mat) {

        int m = mat.size();
        int n = mat[0].size();

        queue<pair<int, int>> q;
        vector<vector<int>> dist(m, vector<int>(n, -1));

        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                if (mat[i][j] == 0) {
                    q.push({i, j});
                    dist[i][j] = 0;
                }
            }
        }

        int dx[4] = {1, -1, 0, 0};
        int dy[4] = {0, 0, 1, -1};
```

```

while (!q.empty()) {

    auto [x, y] = q.front();
    q.pop();

    for (int k = 0; k < 4; k++) {

        int nx = x + dx[k];
        int ny = y + dy[k];

        if (nx >= 0 && nx < m &&
            ny >= 0 && ny < n &&
            dist[nx][ny] == -1) {

            dist[nx][ny] = dist[x][y] + 1;
            q.push({nx, ny});
        }
    }
}

return dist;
}
};

```

Time complexity:

For a grid of size:

$m \times n$

Total nodes: $V = m \times n$

Here,

m = number of rows

n = number of columns

Total number of cells in the grid:

So, each cell = one node

- Every cell is visited only once

- Once visited, we don't visit it again

So total visits: $m \times n$

Because the grid has $m \times n$ cells and each cell is visited once with constant work, the total time complexity is $O(m \times n)$.

Advantage:

1. Optimal time complexity: $O(m \times n)$
2. Naturally models the shortest-path problem, making the logic easy to justify.

Disadvantage:

1. Requires extra queue space of $O(m \times n)$.
2. Implementation may be slightly more complex for beginners unfamiliar with BFS.

Brute Force -

Definition: Brute Force means trying all possible solutions and picking the correct or best one.

Code:

```
class Solution {
public:
    vector<vector<int>> updateMatrix(vector<vector<int>>& mat) {

        int m = mat.size();
        int n = mat[0].size();

        vector<vector<int>> ans(m, vector<int>(n, 0));

        vector<pair<int,int>> dir = {
            {1,0},{-1,0},{0,1},{0,-1}
        };

        for(int i=0;i<m;i++){
            for(int j=0;j<n;j++){

                if(mat[i][j]==0) continue;

                queue<pair<pair<int,int>,int>> q;
                vector<vector<int>> vis(m, vector<int>(n,0));

                q.push({{i,j},0});
                vis[i][j]=1;

                while(!q.empty()){

                    auto node=q.front();
                    q.pop();
```

```

        int x=node.first.first;
        int y=node.first.second;
        int dist=node.second;

        if(mat[x][y]==0){
            ans[i][j]=dist;
            break;
        }

        for(auto d:dir){

            int nx=x+d.first;
            int ny=y+d.second;

            if(nx>=0 && ny>=0 &&
                nx<m && ny<n &&
                !vis[nx][ny]){

                vis[nx][ny]=1;
                q.push({{nx,ny},dist+1});
            }
        }
    }
}
return ans;
}
};

```

Time Complexity:

If grid size = m

Total cells = n

For each cell → BFS takes $O(m \times n)$

Total: $O((m \times n)^2)$

In worst case, BFS may visit all cells:

$O(m \times n)$

Total Work

$(m \times n) \text{ times} \times O(m \times n)$

Final Complexity

$O((m \times n) \times (m \times n)) = O((m \times n)^2)$

Brute force is $O((m \times n)^2)$ because BFS ($O(m \times n)$) is executed for each of the $m \times n$ cells, causing repeated traversal of the grid.

Advantage:

1. Easy to understand and implement — Start BFS from each 1 and find the nearest 0.
2. Guarantees the correct shortest distance because BFS explores level by level.

Disadvantage:

1. Very slow for large matrices since many cells are visited repeatedly.
2. High time complexity: $O((m \times n)^2)$.

Dynamic Programming -

Definition: Dynamic Programming is a technique that solves problems by storing results of subproblems to avoid repeated computation.

Code:

```
class Solution {
public:
    vector<vector<int>> updateMatrix(vector<vector<int>>& mat) {

        int m = mat.size();
        int n = mat[0].size();

        vector<vector<int>> dist(m, vector<int>(n, 100000));

        for(int i=0; i<m; i++){
            for(int j=0; j<n; j++){

                if(mat[i][j]==0)
                    dist[i][j]=0;
                else{

                    if(i>0)
                        dist[i][j]=min(dist[i][j],
                                      dist[i-1][j]+1);

                    if(j>0)
                        dist[i][j]=min(dist[i][j],
                                      dist[i][j-1]+1);
                }
            }
        }
    }
}
```

```

for(int i=m-1;i>=0;i--){
    for(int j=n-1;j>=0;j--){

        if(i<m-1)
            dist[i][j]=min(dist[i][j],
                           dist[i+1][j]+1);

        if(j<n-1)
            dist[i][j]=min(dist[i][j],
                           dist[i][j+1]+1);
    }
}

return dist;
}
};

```

Time Complexity:

Grid size = $m \times n$

Total cells: $m \times n$

Step 1: First Pass (Top $\rightarrow$ Bottom, Left $\rightarrow$ Right)

We go through every cell once

For each cell:

- Check top neighbor

- Check left neighbor

Work per cell = constant = $O(1)$

So total: $O(m \times n)$

Step 2: Second Pass (Bottom $\rightarrow$ Top, Right Left)

Again traverse all cells

For each cell:

- Check bottom neighbor

- Check right neighbor

So: Again constant work $O(m \times n)$

Final Calculation:

$O(m \times n) + O(m \times n) = 2 \times O(m \times n)$

Ignore constant = $O(m \times n)$

We traverse twice in DP to ensure all directional dependencies (left, right, up, down) are properly considered.

And even though we traverse twice, each cell is processed a constant number of times, so overall time complexity remains $O(m \times n)$.

Advantage:

1. Efficient solution with $O(m \times n)$ time complexity.
2. Does not require a queue or graph traversal, making it memory-friendly.

Disadvantage:

1. More difficult to understand than BFS because it relies on two directional passes.
2. Less generalizable to other shortest-path problems.

Why did we choose these 3 approaches:-

For this problem, we selected three approaches to show the progression from a basic solution to the most efficient one. We started with Brute Force BFS, where we run a separate BFS from every cell containing 1 to find the nearest 0. This approach helps us understand that the problem is fundamentally a shortest-distance problem, but it is inefficient because many cells are visited repeatedly. Next, we moved to Dynamic Programming, which utilizes the distances of neighboring cells to compute the answer more efficiently through two passes over the matrix. Finally, we chose Multi-Source BFS, where all 0s are treated as starting points and BFS is performed simultaneously from them. Since BFS naturally finds the shortest distance in an unweighted graph and each cell is visited only once, this becomes the most efficient and intuitive solution.

Problem-2:

It can be solved using three approaches.

1. 0-1 BFS
2. Brute Force DFS
3. Dijkstra's Algorithm

0-1 BFS -

Definition: This approach is designed for graphs whose edge weights are only 0 or 1. It uses a deque to efficiently process nodes and computes the minimum cost path faster than Dijkstra's algorithm for this type of problem.

Code:

```
class Solution {
public:
    int minCost(vector<vector<int>>& grid) {
```

```

int m=grid.size();
int n=grid[0].size();

deque<pair<int,int>> dq;

vector<vector<int>> dist(
    m,vector<int>(n,INT_MAX));

dist[0][0]=0;

dq.push_front({0,0});

vector<pair<int,int>> dir={
    {0,1},
    {0,-1},
    {1,0},
    {-1,0}
};

while(!dq.empty()){

    auto [x,y]=dq.front();
    dq.pop_front();

    for(int i=0;i<4;i++){

        int nx=x+dir[i].first;
        int ny=y+dir[i].second;

        if(nx<0||ny<0||
           nx>=m||ny>=n)
            continue;

        int weight=
            (grid[x][y]==i+1)?0:1;

        if(dist[x][y]+weight
           < dist[nx][ny]){

            dist[nx][ny]=
                dist[x][y]+weight;

            if(weight==0)
                dq.push_front(

```



```

        {nx,ny});
    else
        dq.push_back(
            {nx,ny});
    }
}
}

return dist[m-1][n-1];
}
};

```

Time Complexity:

$O(m \times n)$

How is it derived?

Each cell of the grid is treated as a node:

$V = m \times n$

Each node can have at most 4 neighbors, so:

$E \approx 4V = O(V)$

0-1 BFS is used because every edge weight is either 0 or 1. It uses a deque, where all operations (push_front, push_back, pop_front) take $O(1)$ time.

The complexity of 0-1 BFS is:

$O(V + E)$

Substituting $E = O(V)$:

$O(V + V) = O(V)$

Since $V = m \times n$,

$O(V) = O(m \times n)$

Final Time Complexity

$O(m \times n)$

Unlike Dijkstra's Algorithm, 0-1 BFS does not require a Priority Queue. Because edge weights are restricted to only 0 and 1, a deque can always maintain the correct processing order. This removes the extra $\log V \log V$ factor present in Dijkstra's Algorithm and allows the shortest path to be found in linear time.

Advantage:

1. Most efficient solution with $O(V+E)$ time complexity.
2. Specifically optimized for graphs with edge weights of only 0 and 1.

Disadvantage:

1. Works only for 0-1 weighted graphs, limiting its applicability.
2. The deque-based logic may be unfamiliar to some learners.

Brute Force DFS -

Definition: This approach explores all possible paths from the starting cell to the destination cell using Depth First Search (DFS). It calculates the cost of each path and returns the minimum cost among all possible paths.

Code:

```
class Solution {
public:

    int ans=INT_MAX;

    void dfs(vector<vector<int>>& grid,
            int x,int y,
            int cost,
            vector<vector<int>>& vis){

        int m=grid.size();
        int n=grid[0].size();

        if(x==m-1 && y==n-1){
            ans=min(ans,cost);
            return;
        }

        vis[x][y]=1;

        vector<pair<int,int>> dir={
            {0,1},
            {0,-1},
```

```

        {1,0},
        {-1,0}
    };

    for(int i=0;i<4;i++){

        int nx=x+dir[i].first;
        int ny=y+dir[i].second;

        if(nx>=0 && ny>=0 &&
           nx<m && ny<n &&
           !vis[nx][ny]){

            int extra=
                (grid[x][y]==i+1)?0:1;

            dfs(grid,nx,ny,
                cost+extra,vis);
        }
    }

    vis[x][y]=0;
}

int minCost(vector<vector<int>>& grid) {

    vector<vector<int>> vis(
        grid.size(),
        vector<int>(grid[0].size(),0));

    dfs(grid,0,0,0,vis);

    return ans;
}
};

```

Time Complexity:

$$O(4^{(m \times n)})$$

How is it derived?

Let:

$$N=m \times n$$

where N is the total number of cells in the grid.

From each cell, DFS can potentially move in 4 directions:

- Right
- Left
- Down
- Up

In the worst case, DFS explores almost every possible path. Therefore, at each level of recursion, there can be up to 4 choices.

The recursion tree can have:

$4 \times 4 \times 4 \times \dots (N \text{ times})$

which gives:

$$4^N$$

Since:

$$N = m \times n$$

the final complexity becomes:

$$O(4^{m \times n})$$

DFS tries every possible path from the source to the destination. As the grid size increases, the number of possible paths grows exponentially, making this approach extremely slow for large inputs.

Advantage:

1. Simple concept - Explore all possible paths.
2. Does not require advanced shortest-path algorithms

Disadvantage:

1. Extremely inefficient due to exponential time complexity.
2. Not suitable for large grids because of excessive computation.

Dijkstra's Algorithm -

Definition: This approach models the grid as a weighted graph where each cell is a node. Using a priority queue, it always processes the node with the lowest cost and finds the minimum-cost path from the source to the destination.

Code:

```
class Solution {
```

```

public:
    int minCost(vector<vector<int>>& grid) {

        int m=grid.size();
        int n=grid[0].size();

        priority_queue<
            vector<int>,
            vector<vector<int>>,
            greater<vector<int>>> pq;

        vector<vector<int>> dist(
            m,vector<int>(n,INT_MAX));

        pq.push({0,0,0});
        dist[0][0]=0;

        vector<pair<int,int>> dir={
            {0,1},
            {0,-1},
            {1,0},
            {-1,0}
        };

        while(!pq.empty()){

            auto cur=pq.top();
            pq.pop();

            int cost=cur[0];
            int x=cur[1];
            int y=cur[2];

            for(int i=0;i<4;i++){

                int nx=x+dir[i].first;
                int ny=y+dir[i].second;

                if(nx<0||ny<0||
                    nx>=m||ny>=n)
                    continue;

                int newCost=
                    cost+

```

```

        ((grid[x][y]==i+1)?0:1);

        if(newCost<dist[nx][ny]){

            dist[nx][ny]=newCost;

            pq.push({
                newCost,nx,ny
            });
        }
    }
}

return dist[m-1][n-1];
}
};

```

Time Complexity:

$O((m \times n) \log(m \times n))$

How is it derived?

Step 1: Count the Nodes

Each cell of the grid is treated as a graph node.

Therefore:

$V = m \times n$

where V is the number of vertices.

Step 2: Count the Edges

Each cell can connect to at most 4 neighboring cells.

Thus:

$E \approx 4V$

which means:

$E = O(V)$

Step 3: Apply Dijkstra's Complexity Formula

For a graph implemented with a Min Heap (Priority Queue), Dijkstra's complexity is:

$$O((V+E)\log V)$$

because every heap insertion/removal costs:

$$O(\log V)$$

Step 4: Substitute the Values

Since:

$$E=O(V)$$

we get:

$$O((V+V)\log V)$$

Ignoring constants:

$$O(V\log V)$$

Replacing V with $m \times n$:

$$O((m \times n)\log(m \times n))$$

Dijkstra efficiently finds the shortest path, but every time a node is processed, Priority Queue operations introduce an additional $\log V$ factor.

Advantage:

1. Guaranteed to find the minimum-cost path in weighted graphs.
2. Widely used and applicable to many shortest-path problems.

Disadvantage:

1. Requires a priority queue, increasing implementation complexity.
2. Less efficiency than 0-1 BFS when edge weights are only 0 and 1.

Why did we choose these 3 approaches:-

For this problem, we again followed a progression from a simple approach to an optimized one. We began with DFS/Backtracking, which explores all possible paths from the source to the destination and guarantees finding the answer, but its exponential time complexity makes it impractical for larger inputs. We then used Dijkstra's Algorithm by modeling the grid as a weighted graph, where moving in the indicated direction has a cost of 0 and changing direction has a cost of 1. Since the goal is to find the minimum total cost, Dijkstra provides an efficient shortest-path solution. Finally, we selected 0-1 BFS as the optimal approach because all edge weights are only 0 or 1. By using a deque instead of a priority queue, 0-1 BFS achieves the same result as Dijkstra but with better time complexity, making it the most efficient solution for this problem.

Conclusion :

In both problems, we followed a step-by-step problem-solving approach, starting from a simple idea and gradually moving toward more optimized solutions.

For 01 Matrix, we first used Brute Force BFS to understand the shortest-distance concept, then improved it with Dynamic Programming, and finally arrived at Multi-Source BFS, which provides the most efficient and intuitive solution.

For Minimum Cost to Make at Least One Valid Path in a Grid, we began with DFS to explore all possible paths, then applied Dijkstra's Algorithm by modeling the grid as a weighted graph, and finally optimized it further using 0-1 BFS, which is specifically designed for graphs with edge weights of only 0 and 1.

Overall, these two problems demonstrate how identifying the right graph technique can significantly improve performance. They also show the evolution of BFS from its basic form to advanced variations such as Multi-Source BFS and 0-1 BFS, making them excellent examples of efficient graph-based problem solving.